

To understand the acceptance condition, you have to look at it as a two-part contract between the model and reality.

1. The "Büchi" Condition: $\inf(r) \cap F \neq \emptyset$

This is the standard way to define acceptance for infinite runs. Since the automaton never "stops" in a final state (it runs forever), we can't use the standard FSA rule of "end in a double circle."

Instead, we designate some states as **Accepting (F)**. In your diagram, `free` is an accepting state (noted by the double circle).

- **The Rule:** To be "accepted," a run must pass through an accepting state **infinitely many times**.
- **Concrete meaning:** In your resource manager, a "good" run is one where the resource eventually becomes `free` again and again. if it gets stuck in `occ` or `pend` forever, the run is rejected because the resource is "starved" or "deadlocked."

2. The "Progress" Condition: Timestamps must Diverge

This is the part that specifically kills **Zeno behavior**. It says that for any time T (like 100 years), there must be a step in your sequence where the timestamp t_i is greater than T .

- **Why?** If the timestamps $t_1, t_2, \dots$ converge to a finite number (e.g., $0.9, 0.99, 0.999 \dots \rightarrow 1$), the automaton is "cheating." It is visiting the `free` state infinitely many times, but it's doing it all before 1 second has passed.
- **The Logic:** A run is only valid if **time actually flows to infinity**. If the model can satisfy the "infinite visits" rule only by stuttering infinitely fast in a split second, we ignore that run. It's a "fake" success.

Summary Table: Acceptance Comparison

Feature	Standard FSA	Timed Automata (Infinite)
Input	Finite string	Infinite Timed Word (actions + timestamps)
Success State	Must end in F	Must visit F infinitely often
Time Rule	N/A	Time must diverge ($\sum \Delta t = \infty$)

Putting it together for your Resource Manager

For a sequence of requests to be "accepted" by your resource manager:

1. The resource must return to the `free` state infinitely many times (meaning it never gets permanently stuck).
2. The time between these events must be "real"—you can't just simulate an infinite number of releases in 0.5 seconds.

Think of it this way: The first condition ensures **Fairness** (everyone eventually gets done), and the second condition ensures **Physical Realism** (time doesn't stop).